

Gleaner: A Semantically-Rich and Efficient Online Sampler for Microservice Diagnostics

YIFAN YANG, AOYANG FANG, SONGHAN ZHANG, and PINJIA HE*, The Chinese University of Hong Kong, Shenzhen, China

Distributed tracing in microservices is critical for diagnostics but generates overwhelming data volumes, necessitating intelligent sampling. To maximize fidelity, state-of-the-art (SOTA) tail-based samplers analyze complete (or even log-enriched) traces by modeling them as graphs. However, this reliance on computationally expensive graph analysis creates a performance bottleneck that prohibits their use in online settings.

To this end, we propose Gleaner, an online tail-sampling framework that breaks this trade-off. It is founded on the key insight that explicit graph structures are unnecessary for high-fidelity trace grouping. Instead, Gleaner represents each trace as a "bag-of-edges" augmented with log semantics, replacing slow graph algorithms with highly efficient set-based operations. It also employs an alarm-driven quota and a diversity-preserving strategy to prioritize anomalous and rare traces for downstream Root Cause Analysis (RCA). Experimentally, Gleaner processes traces at 0.74ms each, improving Trace Pattern Coverage by up to 128.7% and Shannon Entropy by up to 32.9% over baselines. At just a 1% sampling rate, Gleaner improves RCA accuracy by 42%-107% over the next-best sampler. Moreover, RCA on Gleaner's sampled data is more accurate than with the entire, unsampled dataset. This result reframes intelligent sampling from a data reduction technique to a powerful signal enhancement paradigm for automated operations.

CCS Concepts: • **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Additional Key Words and Phrases: distributed tracing, trace sampling, microservice system

ACM Reference Format:

Yifan Yang, Aoyang Fang, Songhan Zhang, and Pinjia He. 2018. Gleaner: A Semantically-Rich and Efficient Online Sampler for Microservice Diagnostics. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 21 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Microservice architectures, while enabling rapid development and deployment, introduce profound operational complexity [8, 24, 49, 52]. A single user request can trigger a cascade of interactions across hundreds of services, making system behavior difficult to comprehend [51]. Distributed tracing has emerged as an indispensable tool for observability, providing visibility into these complex request flows [1, 2, 11, 43]. However, the sheer volume of data—often terabytes daily [31]—makes

*Corresponding author.

Authors' Contact Information: Yifan Yang, yifanyang6@link.cuhk.edu.cn; Aoyang Fang, aoyangfang@link.cuhk.edu.cn; Songhan Zhang, 222010549@link.cuhk.edu.cn; Pinjia He, hepinjia@cuhk.edu.cn, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

comprehensive storage and analysis infeasible. Furthermore, trace data typically follows a long-tail distribution, where a few common execution paths dominate. Consequently, intelligent **trace sampling** is not just a practical necessity but a critical component of modern observability pipelines.

The most straightforward approach, **head-based random sampling**, is widely considered inadequate. By making sampling decisions before the execution outcome is known, it disproportionately retains common, healthy traces while being highly likely to discard rare patterns [15]. Since system anomalies and critical failures often manifest as these rare traces, head-based methods risk discarding the very signals that engineers and SREs need most for diagnostics. This fundamental limitation has driven the community towards more sophisticated **tail-based sampling techniques**

olhistie signals uchs tructlure tatus,s the

e(laqa)-20(ffq)-250(ma)-7(dll)-250(trinane)-201aids(infr)-18(nene)-10smaking(t)-250(uitaible)-205(for)-250(hi

We conducted an extensive evaluation on a new, large-scale benchmark dataset. Our results demonstrate that Gleaner achieves high-throughput processing at 0.74ms per trace, while significantly outperforming state-of-the-art samplers on quality metrics. It achieves 11.6%~128.7% improvement in Trace Pattern Coverage and 2.8%~32.9% gains in Shannon Entropy over baselines. At a mere 1% sampling rate, Gleaner boosts the accuracy of downstream RCA tools by 42%~107% over the next-best sampler. Remarkably, RCA on Gleaner's sampled data is more accurate than when using the entire, unsampled dataset. This finding reframes intelligent sampling from a mere data reduction technique to a powerful signal enhancement paradigm for modern automated operations.

The main contributions of this work are as follows:

- We propose a novel method to construct a unified representation from traces and their associated logs, capturing call relationships, internal execution dynamics, and overall anomaly severity.
- We contribute a new large-scale dataset with 161 fault injection cases, over 1.4 million traces, and 517 distinct call paths, which will benefit future research in sampling and diagnostics.
- We present **Gleaner**, the first practical and scalable online sampler that jointly leverages traces and logs, demonstrating its superior performance on an industrial-scale dataset.
- We show that a well-designed sampler not only accelerates downstream RCA tasks but can also significantly improve their accuracy, reframing sampling from a simple data reduction tool to an active signal enhancement strategy for AIOps pipelines.

2 Background and Motivation

This section lays the groundwork for our research. We first establish why intelligent sampling is essential given the data characteristics of distributed tracing (§2.1). We then use concrete examples to expose the critical disconnect between semantically-blind samplers and the demands of modern diagnostic algorithms, which directly motivates the design of Gleaner (§2.2).

2.1 The Challenge of Data Volume in Distributed Tracing

Modern microservice systems generate trace volumes reaching terabytes daily [31], making comprehensive storage infeasible and necessitating intelligent sampling strategies. Critically, trace data exhibits a pronounced long-tail distribution over execution paths, where a *path* is defined as a unique sequence of service operations invoked during request processing. In the Train-Ticket benchmark system [53], the top 24.3% of paths account for 82.3% of total traces. This skew renders simple random sampling ineffective, as it would overwhelmingly retain redundant traces while discarding rare execution patterns. Consequently, intelligent, biased sampling strategies are essential [15, 18, 22, 39]. Our work focuses on tail-based sampling, which makes decisions after trace completion, enabling strategies that prioritize valuable traces based on their characteristics [15, 22, 39].

2.2 The Semantic Gap in Trace Sampling

A critical disconnect has emerged between trace sampling and diagnostics. While state-of-the-art samplers excel at prioritizing traces based on structural rarity or high latency, they remain semantically-blind; they operate on span-level metadata but ignore rich intra-span context like log events [18, 39]. This creates a fundamental problem, as modern diagnostic algorithms are increasingly multi-modal, depending on the correlation between trace structures and log semantics for accurate root cause analysis [10, 44].

Figure 1 provides a concrete example. A trace appears benign based on all span-level metrics (e.g., success status, normal latency) and would be discarded by conventional samplers. However, an embedded ERROR log reveals a critical configuration failure—precisely the kind of subtle issue that

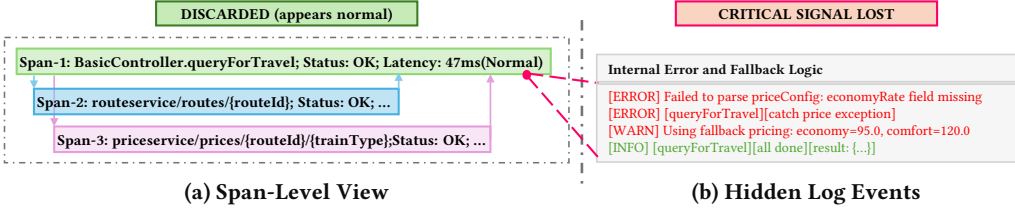


Fig. 1. Example trace illustrating the semantic blind spot of span-centric samplers. Despite normal span-level attributes (success status, acceptable latency), a critical ERROR log within the root span reveals a configuration parsing failure. Traditional samplers would discard this trace as routine, losing diagnostically valuable evidence.

Table 1. Comparison of Trace Sampling Methods and Their Capabilities.

Method	Data Used			Key Capabilities		
	Structure	Attributes	Extra	Anomaly Aware	Intra-span Aware	Online
PEACH [21]	✓					✓
Sifter [22]	✓					✓
Sieve [18]	✓	Latency		Partial		✓
TraceCRL [46]	✓	✓		✓		Offline training
STEAM [15]	✓	✓		✓		Offline training
TraStrainer [17]	✓	✓	Metrics	✓		✓
TracePicker [39]	✓	✓		✓		✓
iTCRL [34]	✓	✓	Log events	✓	✓	Offline training
Gleaner (Ours)	✓	✓	Log events	✓	✓	✓

multi-modal tools are designed to detect. By discarding such traces, the sampler inadvertently filters out the crucial evidence that these downstream tools require, undermining the entire observability pipeline.

Table 1 systematically situates our work and highlights this gap. Existing methods have progressively incorporated more signals, yet only iTCRL and our proposed Gleaner are aware of intra-span log events. Crucially, iTCRL's reliance on offline GNN training makes it unsuitable for real-time use. This analysis highlights the core technical challenge: designing a sampling mechanism that is both semantically rich and computationally tractable for high-throughput, online environments. Gleaner is designed to fill this critical gap.

3 Methodology

This section details Gleaner's design, resolving the trade-off between semantic richness and computational feasibility. We present its architecture (§3.1) and three core components: the **Trace Encoder** (§3.2) transforms raw traces into lightweight yet expressive representations; the **Quota Allocator** (§3.3) implements an adaptive, alarm-driven budget strategy; and the **DPP Selector** (§3.4) selects a final subset rich in anomalous signals and semantic diversity.

3.1 The Gleaner Framework

Gleaner's overall architecture is depicted in Figure 2.

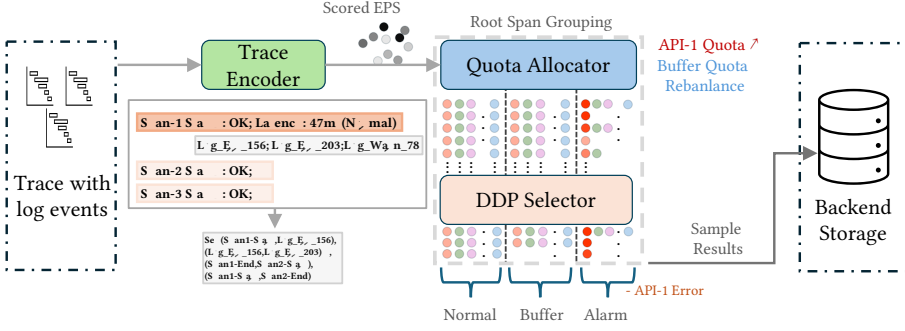


Fig. 2. The three-stage pipeline architecture of Gleaner. First, the **Trace Encoder** consumes raw traces to produce semantic representations and anomaly scores. Next, the **Quota Allocator** adaptively distributes the sampling budget based on real-time monitoring data. Finally, the **DPP Selector** selects a small, diverse, and anomaly-rich subset of traces.

Table 2. Key notation and definitions used in this paper.

Symbol	Definition
$\mathcal{T}$	A distributed trace, represented as a set of spans
$s_i \in \mathcal{T}$	The i -th span in trace $\mathcal{T}$
$L(s_i)$	The set of log events associated with span s_i
$L_w(s_i), L_e(s_i)$	Subsets of $L(s_i)$ with WARN and ERROR levels, respectively
$D(s_i)$	The duration of span s_i
$D_{p90}(G_j)$	The 90th percentile duration for historical traces in group G_j
$E(\mathcal{T})$	The EPS (event-pair set) representation of trace $\mathcal{T}$
$A(\mathcal{T})$	The anomaly score of trace $\mathcal{T}$
$\mathcal{B}$	The total sampling budget (number of traces to select)
$\mathcal{G}$	The set of root span groups: $\{G_1, G_2, \dots, G_k\}$
G_j	A group of traces sharing the same root span (API endpoint)
q_j	The sampling quota allocated to group G_j
$\mathcal{A}$	The set of alerted APIs from external monitoring systems
L	The DPP kernel matrix used by the DPP Selector
S	The final sampled subset of traces output by Gleaner

3.2 Trace Encoder: A Lightweight Semantic Representation

Inspired by the event-centric view adopted by multimodal RCA systems such as Nezha [44] and iTCRL [34], we re-conceptualize a trace from a graph of spans into a **set of event pairs (EPS)** that captures both inter-span calls and intra-span log details. Span lifecycle markers and salient in-span events are treated as the basic diagnostic units, but instead of organizing them in a graph, Gleaner encodes them as local event pairs for online similarity computation. This representation is deterministic and robust to asynchronous timing variations, semantically rich enough to encode both structure and log content, yet remains computationally tractable through a lightweight, hashable data structure that enables efficient similarity computation. We define a **trace pattern** as a unique EPS: two traces sharing the same EPS belong to the same pattern, representing identical system behavior despite potentially different trace IDs or timestamps.

Algorithm 1 Trace Encoder

Require: Raw trace $\mathcal{T}$ with spans and logs

Ensure: EPS $E(\mathcal{T})$, anomaly score $A(\mathcal{T})$

```

1:  $E \leftarrow \emptyset, A \leftarrow 0$ 
2: Compute  $A$  via Eq. 1 ▷ Errors, logs, latency
3: for  $s_i \in \mathcal{T}$  do ▷ Intra-span encoding
4:    $\mathcal{E}_i \leftarrow [\text{span\_start}, \text{logs}_{\text{sorted}}, \text{status\_error}, \text{perf\_deg}, \text{span\_end}]$ 
5:    $E \leftarrow E \cup \{(e_k, e_{k+1}) \mid e_k, e_{k+1} \in \mathcal{E}_i\}$  ▷ Bi-grams
6: end for
7: for  $(s_p, s_c)$  in parent-child pairs do ▷ Inter-span linking
8:    $E \leftarrow E \cup \{(\text{end}_{s_p}, \text{start}_{s_c})\}$ 
9: end for
10: return  $(E, A)$  ▷ Set  $E$  auto-deduplicates

```

Our encoder converts each trace $\mathcal{T}$ into a tuple $(E(\mathcal{T}), A(\mathcal{T}))$ as outlined in Algorithm 1. For each span s_i , we construct a canonical event sequence and apply bi-gram encoding to capture execution flow. For example, the root span in Figure 1 with events $\langle \text{start_basic}, \text{log_e_156}, \text{log_e_203}, \text{log_w_78}, \text{end_basic} \rangle$ produces intra-span pairs $\{(\text{start_basic}, \text{log_e_156}), (\text{log_e_156}, \text{log_e_203}), (\text{log_e_203}, \text{log_w_78}), (\text{log_w_78}, \text{end_basic})\}$. Inter-span relationships link parent-end to child-start events, such as $(\text{end_basic}, \text{start_route})$. The resulting set $E(\mathcal{T})$ automatically deduplicates redundant pairs from parallel calls.

Event ID Management and Anomaly Scoring. We maintain an Event Manager that assigns unique integer IDs to different event types. For log events, we apply Drain [14] during collection to strip dynamic variables and map recurring messages to stable templates; we then use the template ID as the event ID. This normalization is essential for preventing state-space explosion from high-cardinality log fields while preserving recurring event semantics. Span start, span end, status error, and performance degradation events are dynamically assigned IDs based on the service_name + span_name combination they belong to. Concurrently, we calculate a fine-grained anomaly score $A(\mathcal{T})$ for each trace. This score is a weighted sum of indicators from span status, log levels, and performance degradation, as defined in Equation 1:

$$A(\mathcal{T}) = \sum_{s_i \in \mathcal{T}} (w_{err} \cdot \mathbb{I}_{err}(s_i) + w_{lw} \cdot |L_w(s_i)| + w_{le} \cdot |L_e(s_i)|) + A_{perf}(\mathcal{T}) \quad (1)$$

where w_{err}, w_{lw}, w_{le} are configurable weights (in this paper, we set $w_{err}=5, w_{lw}=1, w_{le}=2$); $A_{perf}(\mathcal{T})$ is added only if end-to-end latency exceeds $1.2 \times D_{p90}(G_j)$ for the root-span group G_j .

Hyperparameter Robustness. The exact form of $A(\mathcal{T})$ is not central to our method; its role is to provide a lightweight relevance prior that separates clearly anomalous traces from the healthy majority. In our benchmark evaluation, varying w_{err}, w_{lw}, w_{le} and the latency score term $A_{perf}(\mathcal{T})$ leaves coverage and entropy metrics completely unchanged, with less than 1% variance only in the capture rate of diagnostically relevant traces. Empirically, our default setting prioritizes hard failures like status errors over soft signals like logs, mirroring standard SRE triage principles. We emphasize that these parameters remain configurable, allowing operators to dynamically shift focus, such as making $A_{perf}(\mathcal{T})$ more aggressive to “zoom in” on performance regressions during tuning campaigns.

Intra-Span Canonical Ordering. For each span s_i , we construct a canonical event sequence immune to timing variations common in asynchronous systems. Gleaner enforces a deterministic sequence: (1) span start, (2) log events ordered by timestamp, (3) anomaly indicators if present

(errors, performance degradation), and (4) span end. We then apply bi-gram encoding to capture the span's internal execution flow as event pairs.

Inter-Span Structural Linking. We extract parent-child span relationships and create event pairs linking the parent's `span_end` event to the child's `span_start` event. Rather than building a complete call tree, we use this pair-based approach for robustness against *broken traces* common in production, where some spans may have missing parent references due to instrumentation gaps or collection failures.

Set-Based Representation. All generated event pairs are collected into a single set, $E(\mathcal{T})$. This set-based representation is a critical design choice, as it automatically deduplicates redundant pairs that arise from fan-out parallel calls, a common scenario in microservices.

Insight 1: Modeling a trace as a hashable EPS captures both inter-span structure and intra-span semantics while eliminating parallel duplication and enabling efficient Jaccard similarity calculation.

3.3 Quota Allocator: Alarm-Driven Adaptive Budgeting

Production samplers must respond to incidents like SREs do, but existing methods are disconnected from external monitoring, fail to adapt during traffic drops, and risk over-sampling during alarm storms [39]. Our allocator integrates with monitoring systems to dynamically adjust sampling budgets, scaling up when failures cause traffic drops while using caps to prevent any single alarm from exhausting resources.

Alarm Generation and Grouping Strategy. Gleaner groups traces at the **root span** level, where the root span is the first span of a trace representing the entry point API. This grouping aligns with how production monitoring organizes SLIs for latency and error rate. Root spans provide a stable semantic anchor while enabling efficient downstream processing: endpoints exhibit heterogeneous diversity yet maintain high intra-group homogeneity, as validated in Figure 3.

We deliberately avoid heavyweight GNN-based clustering, which incurs prohibitive overhead for online streams, and structural hashing like path codes [39], which fragments quota allocations due to spurious groups from minor control-flow variations. Figure 3 shows that while endpoints contribute heterogeneously to total unique paths, they exhibit very high intra-group similarity with path similarity often exceeding 0.9. This confirms root-span grouping correctly identifies the primary semantic axis while filtering artificial structural diversity, producing compact, homogeneous groups required for efficient DPP selection.

Two-Layer Allocation Process. Gleaner maintains a time-based *buffer* that continuously stores recent traces. When an external alarm is triggered, this buffer provides the *normal period* baseline, while incoming traces during the active alarm constitute the *abnormal period*. Our allocation mechanism operates as a two-layer process over these two periods.

Layer 1: Global Budget Adjustment. First, we manage the total sampling budget $\mathcal{B}$ to adapt to real-time traffic conditions. Severe system failures often cause sharp traffic volume (QPM) drops due to cascading effects, requiring budget scaling to preserve coverage. When an alarm is triggered, we compare the QPM of the current abnormal period to the historical average from the buffer. If a significant drop is detected, a scaling factor is applied to temporarily increase the total budget $\mathcal{B}$. This adjusted budget is then allocated between the normal and abnormal periods based on their relative QPM, prioritizing the incident period.

Layer 2: Intra-Group Allocation. Second, the budget for each period is allocated to the root span groups $G_j \in \mathcal{G}$ within it. For the normal period, the budget is distributed evenly across all groups to maximize baseline diversity and maintain comprehensive system visibility. For the abnormal period, we mimic an SRE's focus: groups G_j corresponding to alerted APIs in $\mathcal{A}$ receive a **boosted quota**

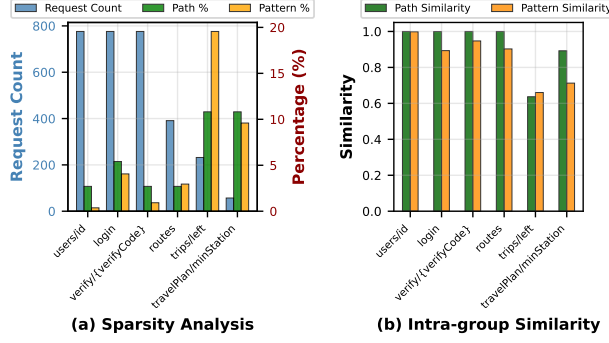


Fig. 3. Root span grouping validation on Train-Ticket benchmark. (a) Sparsity analysis: ratio between request volume (QPM) and contribution to total unique paths/patterns for selected API endpoints. (b) Intra-group similarity: average Jaccard similarity of paths and event-pair patterns within each group. High intra-group similarity validates homogeneity for efficient DPP optimization.

q_j , potentially up to 3x the average. This boost is subject to a **cap** (e.g., 50% of the period's budget) to prevent any single issue from exhausting resources. The remaining budget is then distributed among non-alarmed groups, guaranteeing baseline coverage for all services (detailed coverage metrics in §4). When no alarms are active, the allocator processes the buffer uniformly, preserving system-wide visibility without alarm-driven prioritization.

Insight 2: An effective online sampler should **mimic an SRE's focus during an incident**. By integrating with external alarms and dynamically rebalancing budget to compensate for traffic drops, the sampler can prioritize the most relevant data when it is needed most.

3.4 DPP Selector: Optimized Selection for Diversity and Relevance

We employ Determinantal Point Processes (DPP) [4] to select diverse, high-quality trace subsets. The DPP kernel matrix L encodes anomaly scores $A(\mathcal{T})$ on its diagonal and EPS Jaccard similarities on off-diagonal entries. Standard DPP is computationally expensive; we introduce two optimizations for online deployment.

Early Termination via High Intra-Group Similarity. Root-span grouping creates semantically coherent groups by construction—all traces in group G_j originate from the same API endpoint. This homogeneity (Figure 3b shows intra-group Jaccard similarity often exceeding 0.9) enables fast greedy DPP [4] to converge rapidly, as marginal diversity gains drop below threshold ϵ after selecting few candidates.

Persistent Cross-Batch Similarity Caching. EPS patterns exhibit high temporal stability—the same patterns recur as users repeatedly invoke API endpoints. We implement a global LRU cache keyed by $\text{hash}(E(\mathcal{T}_i), E(\mathcal{T}_j))$ that persists across batches. This achieves a 91.8% hit rate, dramatically reducing redundant Jaccard similarity calculations with negligible memory overhead.

Insight 3: The prohibitive cost of DPPs can be overcome by exploiting the **high intra-group similarity unique to our design**. Unlike post-hoc clustering methods, our root-span grouping produces semantically coherent groups by construction, enabling fast greedy DPP [4] to converge rapidly. Combined with persistent cross-batch similarity caching, this makes DPP practical for high-throughput online streams.

Table 3. Distribution of fault injection cases in Dataset $\mathcal{A}$.

	App	Network	HTTP	Container
Type	CPU/mem code	Delays, loss, partitions	Req delays, aborts, tamper	Container kills
Count	46	39	58	18

Table 4. Statistics of the benchmark datasets used in our evaluation.

Metric	Dataset $\mathcal{A}$	Dataset $\mathcal{B}$ (from TracePicker)				
	(Ours)	Train-Ticket	Media	OnlineBoutique	Sock-Shop	Social-Network
Total Traces	1,474,537	22,000	28,710	42,074	43,472	32,123
Total Trace Patterns	285,059	2,937	3,841	4,216	271	2,443
Total Entries	24	16	2	6	8	1

These optimizations make DPP practical for high-throughput online streams, enabling diversity-aware selection at sub-millisecond latency.

4 Evaluation

We evaluate Gleaner to answer four key research questions:

- **RQ1:** How does Gleaner compare to state-of-the-art samplers in sampling quality, diversity, and anomaly capture?
- **RQ2:** What is the contribution of each design component in Gleaner's overall performance?
- **RQ3:** Does Gleaner improve downstream root cause analysis (RCA) accuracy?
- **RQ4:** What is Gleaner's efficiency in terms of runtime overhead, budget control, and information density?

4.1 Experimental Setup

Benchmark Datasets. To ensure a comprehensive and robust evaluation, we use two distinct sets of benchmarks, which we denote as Dataset $\mathcal{A}$ and Dataset $\mathcal{B}$.

1) *In-depth Fault Analysis Dataset (Dataset $\mathcal{A}$):* Our primary evaluation is performed on a new, large-scale dataset we generated from the Train-Ticket [53] benchmark. To rigorously test sampler performance under failure scenarios, we collected 161 fault injection cases using ChaosMesh [3], covering 16 types of production-realistic failures across application, network, HTTP, and container layers. This dataset, **Dataset $\mathcal{A}$** , allows for in-depth analysis of anomaly capture and its impact on downstream RCA.

2) *Cross-System Generalization Dataset (Dataset $\mathcal{B}$):* To evaluate Gleaner's generalizability, we use a public dataset from TracePicker [39]. This collection, **Dataset $\mathcal{B}$** , contains traces from $\square$ ve different well-known microservice systems, allowing us to assess performance across diverse architectures and workloads. Since these datasets consist only of traces without corresponding logs or alarm labels, we use a variant of Gleaner (Gleaner w/o Logs w/o Alarms) for this part of the evaluation.

Baseline Methods. We compare Gleaner against a suite of representative and state-of-the-art samplers:

- **Random:** Uniformly samples traces at a $\square$ xed probability.
- **Sieve** [18]: An online sampler that prioritizes structurally and temporally rare traces using Robust Random Cut Forest (RRCF).
- **Sifter** [22]: An online sampler that models common trace structures and prioritizes those that deviate from the model.
- **TraStrainer** [17]: An adaptive sampler that correlates traces with external system metrics. We provide it with key metrics (CPU, memory, P50/P90 latency) as input. We also include a **TraStrainer w/o Metrics** variant to assess its trace-only performance.
- **TracePicker** [39]: A state-of-the-art online sampler that uses an optimization-based approach to prioritize anomalous traces and then maximize path coverage.

Implementation Details. All experiments were run on AMD EPYC 9754 128-Core Processors (2 cores per sampler). For samplers with imprecise budget control, we apply standard normalization [17, 22]. For baseline methods, we use publicly released code where available, and for methods without public implementations, we contacted the related authors and strictly followed their published specifications. All metrics are computed independently for each of the 161 fault injection cases and then averaged to reduce variance from stochastic factors.

4.2 Evaluation Metrics

We evaluate Gleaner’s performance across two dimensions: intrinsic sampling quality in RQ1, RQ2, and RQ4, and downstream task effectiveness in RQ3.

For sampling quality, we adopt established metrics from prior work [15, 17, 34, 39]. Following prior work, we organize them into three complementary families: *coverage* measures how much of the original observability space remains after sampling, *entropy* measures whether the sampled traces preserve sufficient information density rather than collapsing onto a few dominant patterns, and *proportion* measures whether the sampler is biased toward diagnostically valuable traces.

- **Coverage metrics:** We report **API Coverage**, **Path Coverage**, and **Trace Pattern Coverage**. API Coverage measures whether the sampled set preserves visibility across business entry points. Path Coverage measures whether distinct execution flows are retained; in our implementation, this uses a deduplicated path encoding, where exact parallel duplicates at the same level are removed before path comparison. Trace Pattern Coverage (as defined in §3.2) measures how many EPS-level behavioral patterns remain after sampling.
- **Entropy metric:** We report **Shannon Entropy** to quantify whether sampled traces are broadly distributed across patterns, rather than being concentrated in a few dominant behaviors.
- **Proportion metrics:** We report **Proportion Anomaly** and **Proportion Rare** to quantify whether the sampler preferentially preserves diagnostically valuable traces from the anomaly and long-tail perspectives, respectively.
- **Downstream utility metric:** For RCA evaluation, we use **Accuracy@k**, the proportion of cases where the true root cause is ranked in the top-k results.

For efficiency analysis, we measure **Runtime Per Trace** and **Actual SForot cauoe er dominant pattern**

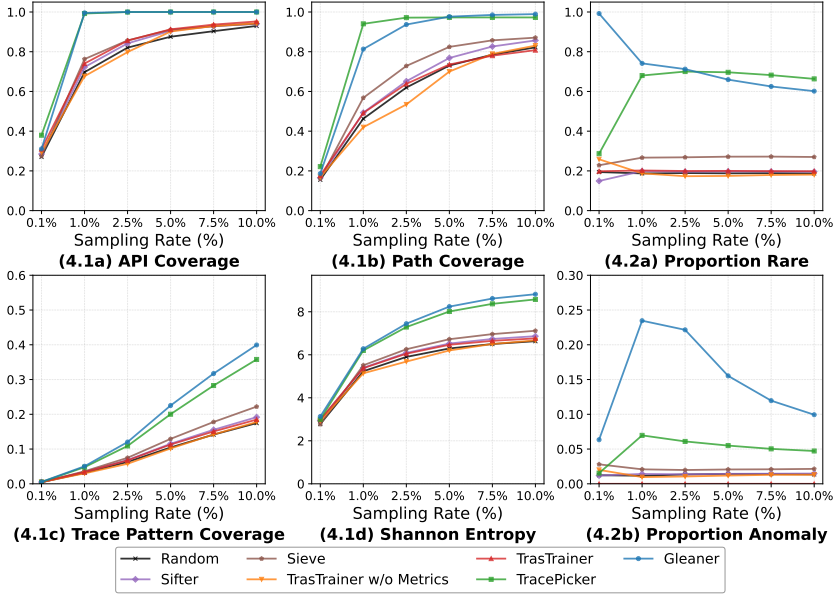


Fig. 4. Sampling quality evaluation on Dataset $\mathcal{A}$. Left two columns (4.1a-d): Coverage and diversity metrics showing Gleaner’s consistent superiority across all dimensions. Rightmost column (4.2a-b): Anomaly and rarity capture demonstrating Gleaner’s dramatic advantage in prioritizing diagnostically relevant traces.

of 0.992, 3.5 $\times$ higher than TracePicker. For Proportion Anomaly, Gleaner maintains 2.1–8.3 $\times$ improvement over baselines across all rates, validating alarm-driven quota allocation.

Cross-System Generalization on Dataset $\mathcal{B}$. Figure 5 shows Gleaner (w/o Logs w/o Alarms) consistently outperforms baselines across $\square$ ve microservice benchmarks. At 10% sampling rate, Gleaner achieves average Trace Pattern Coverage of 0.689, outperforming TracePicker by 0.088 and other methods by 2.8–4.0 $\times$. The main exception is SockShop, whose workload is dominated by a single front-end entry point. Because Gleaner’s allocator groups traces by root span, this architecture largely removes the grouping advantage and makes the method behave closer to a structure-driven diversity sampler; even under this unfavorable condition, Gleaner remains competitive with TracePicker.

Finding 1: Gleaner produces a sample set that is more comprehensive, diverse, and diagnostically relevant than state-of-the-art baselines on Dataset $\mathcal{A}$, and generalizes well across systems in Dataset $\mathcal{B}$. Its gains are largest in systems with multiple business entry points and richer in-span events; when those conditions weaken, as in SockShop, Gleaner degrades gracefully toward a structure-driven diversity sampler while remaining competitive.

4.4 RQ2: Ablation Study

To understand the contribution of Gleaner’s core components and design choices, we conduct an ablation study. Table 5 summarizes all variants. We report results in two groups: Group 1 examines the impact of data sources and representation methods, and Group 2 explores the balance between sampling strategies. All variants are evaluated on Dataset $\mathcal{A}$ using the same metrics as RQ1.

Group 1: Component Effectiveness (Figure 6). This group validates the impact of input signals and structural representation.

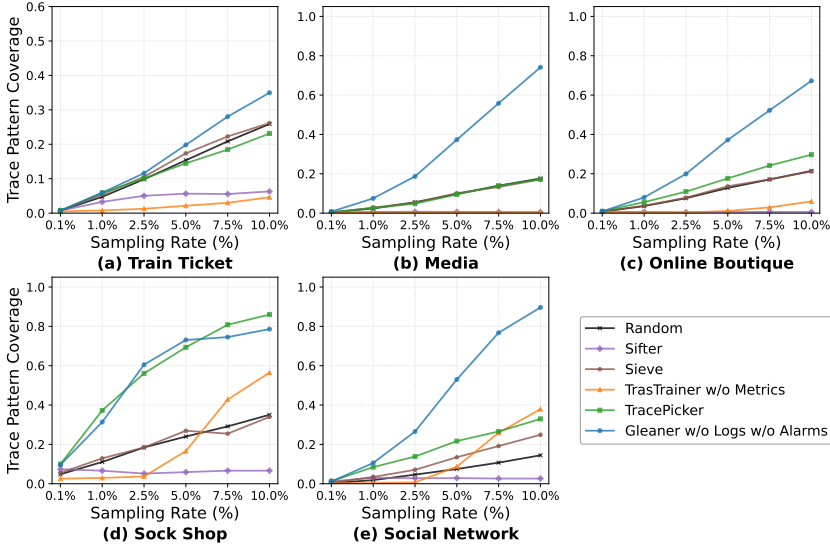


Fig. 5. Cross-system evaluation on Dataset $\mathcal{B}$ (5 microservice benchmarks). Gleaner consistently outperforms baselines across most systems, with comparable performance to TracePicker on simpler architectures.

Table 5. Summary of Gleaner variants designed for ablation study. The table disentangles the contribution of input signals, representation methods, and sampling strategies.

Variant Name	Input Signals		Representation	Sampling Strategy			Motivation / Focus
	Logs	Alarms	Structure Enc.	Grouping	Anomaly	Diversity	
Gleaner	✓	✓	EPS	✓	✓	✓	Proposed Method (Full)
<i>Group 1: Inputs & Rep.</i>							
w/o Logs		✓	EPS	✓	✓	✓	Impact of semantic richness
w/o Alarms	✓		EPS	✓	✓	✓	Impact of dynamic quota
w/o Logs & Alarms			EPS	✓	✓	✓	Baseline structural performance
WL Kernel	✓	✓	Graph	✓	✓	✓	Efficiency check (EPS vs. Graph)
<i>Group 2: Strategies</i>							
Pure Diversity	✓	✓	EPS			✓	Impact of Grouping & Anomaly
Pure Anomaly	✓	✓	EPS		✓		Impact of Grouping & Diversity
w/o Anomaly	✓	✓	EPS	✓		✓	Impact of Anomaly prioritization
w/o Diversity	✓	✓	EPS	✓	✓		Impact of Diversity selection

Overall, Group 1 shows that the structural signal captured by our Event Pair Set (EPS) representation provides a strong foundation, while logs and alarms improve fault relevance. Even without logs and alarms, the sampler remains consistently better than Random across the quality metrics, indicating that EPS alone already captures useful execution regularities. Counter-intuitively, replacing EPS with the WL Graph Kernel yields *lower* diversity: pattern coverage drops from 39.9% to 32.8% while runtime increases 8.4× (quantified in Table 8). This suggests that exact structural matching is too rigid for microservices, where concurrent fan-out patterns create structurally distinct graphs for identical logical requests; EPS is more robust to these concurrency-induced variations. Finally, disabling alarms reveals an expected trade-off: proportion anomaly drops significantly, while entropy and coverage metrics slightly increase. This occurs because without alarm-driven

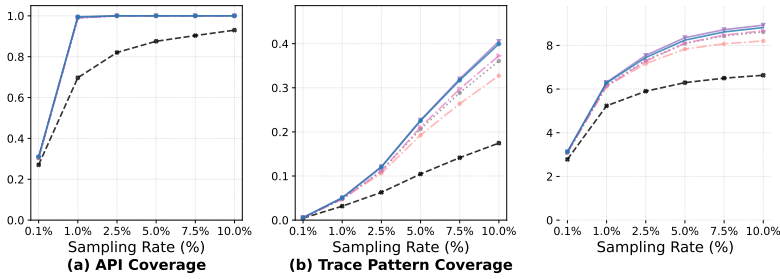


Fig. 6. Ablation study Group 1: Impact of semantic components (logs, alarms) and structural representation (vs. WL Kernel) on Dataset $\mathcal{A}$.

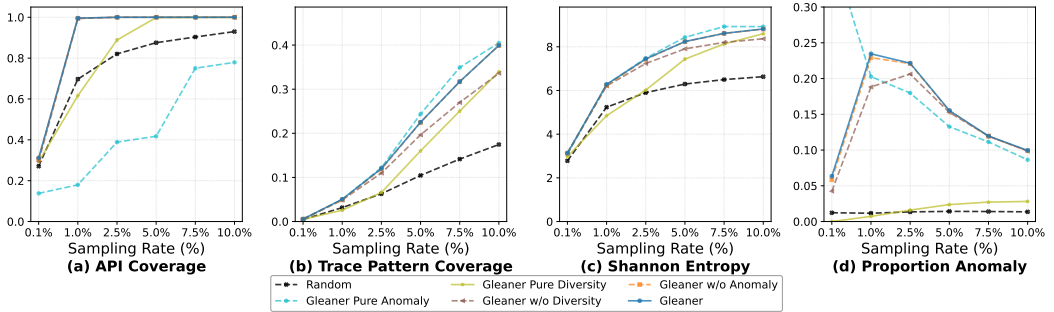


Fig. 7. Ablation study Group 2: Performance comparison of different sampling strategies, highlighting the synergy between diversity and anomaly scoring on Dataset $\mathcal{A}$.

quota allocation, the budget distributes more uniformly across all API groups than concentrating on alarm groups, leading to marginally higher diversity at the cost of reduced fault relevance.

Group 2: Strategy Balance (Figure 7). This group explores the necessity of balancing anomaly prioritization with diversity by examining extreme strategies.

Group 2 demonstrates that each component is necessary: removing any single mechanism degrades the corresponding quality dimension. Most notably, *Pure Anomaly* exhibits an interesting trade-off: while it achieves higher entropy and pattern coverage than the full model, API coverage collapses at low rates (1%: 0.18 vs Random's 0.70); without grouping, the sampler fixates on whichever endpoints currently exhibit anomalies in each batch, leading to highly unbalanced API representation. Meanwhile, *Pure Diversity* shows that removing root-span grouping causes coverage to drop significantly: without per-group batching, different batches repeatedly select the same globally diverse traces, reducing overall coverage. The intermediate variant *w/o Diversity* maintains most metrics but suffers reduced pattern representativeness, while *w/o Anomaly* performs nearly identically to the full model except for slightly lower anomaly capture. Overall, the full *Gleaner* achieves the optimal balance across all dimensions.

Table 6. RCA accuracy comparison on Dataset $\mathcal{A}$. Unsampled row shows performance using all traces. Gleaner achieves the highest accuracy across all tools.

Sampler	MicroRCA				Nezha				ShapleyIQ			
	AC@1		AC@3		AC@1		AC@3		AC@1		AC@3	
	1%	10%	1%	10%	1%	10%	1%	10%	1%	10%	1%	10%
Random	0.39	0.45	0.44	0.49	0.06	0.09	0.15	0.19	0.17	0.40	0.30	0.56
TracePicker	0.45	0.45	0.49	0.49	0.08	0.06	0.19	0.20	0.26	0.39	0.45	0.48
Sieve	0.42	0.45	0.47	0.49	0.11	0.09	0.23	0.17	0.27	0.47	0.40	0.60
Sifter	0.41	0.45	0.48	0.49	0.02	0.08	0.15	0.25	0.18	0.40	0.37	0.58
TraStrainer	0.40	0.45	0.49	0.49	0.10	0.07	0.24	0.19	0.06	0.03	0.19	0.22
TraStrainer w/o M	0.38	0.45	0.45	0.50	0.09	0.09	0.26	0.24	0.09	0.32	0.23	0.57
Gleaner	0.45	0.45	0.50	0.50	0.19	0.14	0.45	0.35	0.56	0.52	0.64	0.61
Unsampled	0.45		0.50		0.11		0.27		0.41		0.57	

Table 7. Ablation analysis on RCA accuracy. The synergistic combination of anomaly, grouping, and diversity outperforms any single strategy.

Sampler	Nezha				ShapleyIQ			
	AC@1		AC@3		AC@1		AC@3	
	1%	10%	1%	10%	1%	10%	1%	10%
Pure Anomaly	0.03	0.13	0.12	0.24	0.35	0.49	0.43	0.59
Pure Diversity	0.05	0.09	0.14	0.22	0.11	0.47	0.21	0.58
w/o Diversity	0.10	0.07	0.23	0.19	0.45	0.49	0.60	0.60
w/o Anomaly	0.14	0.11	0.29	0.24	0.57	0.52	0.65	0.60
w/o Alarms	0.12	0.13	0.32	0.31	0.42	0.45	0.58	0.60
w/o Logs	0.13	0.11	0.32	0.27	0.50	0.51	0.62	0.62
w/o Logs w/o Alarms	0.14	0.13	0.28	0.30	0.45	0.48	0.58	0.61
Gleaner	0.19	0.14	0.45	0.35	0.56	0.52	0.64	0.61
Unsampled	0.11		0.27		0.41		0.57	

Finding 2: Ablations confirm the role of each design component. EPS provides a strong structural foundation and is more effective than the Graph Kernel alternative. Logs and alarms improve fault relevance, and removing alarms induces the expected trade-off between anomaly focus and diversity. Root-span grouping, anomaly prioritization, and diversity selection are all necessary to achieve optimal sampling quality.

4.5 RQ3: Impact on Downstream Root Cause Analysis

We evaluate how sampler choice affects automated Root Cause Analysis (RCA) accuracy using three open-source RCA algorithms: **MicroRCA** [38], **Nezha** [44], and **ShapleyIQ** [25]. For fair comparison, we provide aggregated metrics (p90 latency, error count) to supporting tools. Results are in Table 6.

Table 6 shows Gleaner's substantial impact on RCA accuracy. At 1% sampling rate, Gleaner achieves +107% for Nezha (AC@1: 0.19 vs. 0.11), +42% for Nezha (AC@3: 0.45 vs. 0.26), and +107% for ShapleyIQ (AC@1: 0.56 vs. 0.27) compared to next-best samplers.

Most strikingly, Gleaner's 1% sample enables RCA tools to *outperform the complete unsampled dataset*: ShapleyIQ achieves +36.6% AC@1 (0.56 vs. 0.41), Nezha achieves +72.7% AC@1 (0.19 vs. 0.11) and +66.7% AC@3 (0.45 vs. 0.27), while MicroRCA maintains equivalent accuracy with 99% less data. This reveals that strategic curation enhances signal quality beyond quantity by concentrating diagnostically-rich signals and filtering redundant traces.

Why does sampling outperform full data? This result runs counter to the intuition that "more data is better." We investigated this by expanding our evaluation to over 10 additional RCA algorithms. We found that algorithms like MicroRCA were notably insensitive to sampling strategies. This insensitivity arises because MicroRCA locates root causes by analyzing Pearson correlations between aggregated service metrics, such as P90 latency, and system metrics like CPU usage on an attribute graph. Since accurate statistical features are provided as inputs and the service topology is preserved, its Personalized PageRank algorithm remains stable regardless of specific trace selection. In contrast, other tools like DiagFusion [48] and EADRO [23] failed to localize faults in our complex dataset regardless of data volume, achieving less than 10% accuracy. Thus, we present MicroRCA as a stability baseline, while focusing on Nezha and ShapleyIQ to demonstrate the impact of data quality.

To pinpoint the source of this performance gain, we conducted a detailed ablation study on Gleaner's components (Table 7). We first observe that disjoint sampling strategies, such as *Pure Anomaly*, *Pure Diversity*, and *w/o Diversity*, consistently fail to surpass the full dataset baseline. In fact, without the synergy of components, these ablated variants often yield results inferior to

Table 8. Efficiency comparison on Dataset $\mathcal{A}$ at 5% target sampling rate. Gleaner achieves optimal balance across all three efficiency dimensions.

Sampler	Runtime (ms)	Benefit-Cost Ratio	Actual Rate (%)
Gleaner	0.741	0.81	5.0
Gleaner WL Kernel	6.251	0.73	5.0
TracePicker	1.008	0.76	5.0
Sieve	0.629	0.33	33.2
Sifter	1.220	0.21	94.3
TraStrainer	69.857	0.43	11.2
TraStrainer w/o M	1.349	0.17	72.0
Random	0.008	0.41	5.0

even simple Random sampling. This indicates that neither anomaly filtering nor structural diversity alone is sufficient; their synergy is essential. Beyond this general failure, the results reveal that different RCA principles demand different data properties.

For causal-inference-based tools like **ShapleyIQ**, structural integrity is paramount. When we remove grouping in the *Pure Diversity* variant, ShapleyIQ’s AC@3 at 1% sampling rate collapses from 0.64 to 0.21. In contrast, removing anomaly weighting in the *w/o Anomaly* variant has little impact, maintaining a high AC@3 of 0.65 compared to 0.64 for the full model. This behavior occurs because ShapleyIQ relies on comparing normal and abnormal executions to calculate “marginal contributions.” By enforcing uniform sampling across API endpoints through grouping, Gleaner ensures a complete “healthy baseline” for every endpoint, preventing the missing-baseline failures that cripple the ungrouped *Pure Diversity* variant.

Conversely, statistical-pattern-based tools like **Nezha** depend heavily on anomaly-guided diversity. Disjoint strategies fail dramatically here: for instance, *Pure Anomaly* (0.03), *Pure Diversity* (0.05), and *w/o Diversity* (0.10) all fall far below the *Unsampled* baseline (0.11) in AC@1 at 1% rate. Notably, *Pure Anomaly* and *Pure Diversity* even underperform Random sampling (0.06). Similarly, the *w/o Anomaly* variant, which provides diversity without biasing, degrades significantly as AC@3 drops from 0.45 to 0.29. This specific sensitivity indicates that Nezha, which contrasts event pattern frequencies, requires a dataset that is both diverse to cover various failure modes and anomaly-biased to enhance the signal-to-noise ratio. Furthermore, the inclusion of *log encoding* captures failures manifested only in logs, providing the necessary multimodal evidence for complex faults.

Finding 3: Gleaner consistently improves downstream RCA accuracy and can even outperform using the full dataset at low sampling rates. The gain comes from curating diagnostically rich traces: grouping preserves healthy baselines for causal inference, and anomaly-guided diversity and log encoding strengthen the signal for pattern-based tools.

4.6 RQ4: Efficiency Analysis

This RQ assesses sampler efficiency under realistic online conditions with a 5% target sampling rate. To accurately reflect real-world deployment scenarios, we do not artificially constrain the budget of non-deterministic samplers, allowing their effective sampling rates to be governed by their own algorithms. The results in Table 8 reveal significant differences in practical deployability.

Gleaner’s efficiency profile (Table 8) validates the practical viability of our optimizations: persistent cross-batch similarity caching and early termination enable 0.741ms per-trace latency (26.5% faster than TracePicker, 94× faster than TraStrainer) while achieving the highest benefit-cost ratio

of 0.81. Moreover, Gleaner maintains strict budget control with exactly 5.0% sampling rate, in contrast to non-deterministic samplers that exhibit severe overruns up to 18.86 $\times$ over budget.

We additionally profiled the memory footprint of our current Python prototype. Processing 88,266 spans in one run yielded a peak memory usage of approximately 1.3GB. Given the data volume involved, this result suggests that the prototype-level implementation remains within a practical range for a dedicated sampler process.

The comparison with *Gleaner WL Kernel* directly validates our EPS design choices: Gleaner achieves both a higher benefit-cost ratio (0.81 vs. 0.73) and much lower latency (0.741ms vs. 6.251ms, 8.4 $\times$ faster). This provides strong evidence that Event Pair Set retains the structural signal needed for diversity optimization while substantially reducing computational overhead.

Finding 4: Gleaner achieves production-friendly efficiency with strict budget control, low per-trace latency, and high information density. Compared to an exact graph-kernel variant, EPS is both faster and yields higher benefit-cost ratio, directly validating the effectiveness of our representation and engineering optimizations.

5 Discussion

5.1 Applicability

Gleaner is most effective in microservice systems with multiple business entry points and relatively rich in-span events. Multiple entry points make root-span grouping more informative, while logs and span events provide the semantic signals needed to distinguish traces that are structurally similar but diagnostically different. This observation also suggests two practical considerations for deployment: root spans should reflect business entry points as clearly as possible, and trace-log correlation should be maintained consistently so that diagnostic signals remain available to the sampler.

When these conditions are weaker, Gleaner's advantages become smaller but do not disappear. In systems dominated by a single entry point, such as SockShop, root-span grouping provides less leverage and the method behaves closer to a structure-driven diversity sampler. Likewise, when logs and alarms are both unavailable, Gleaner falls back to its structural encoding and remains competitive with strong trace-only baselines, as shown in Figure 5. Overall, Gleaner benefits most from the combination of structural diversity and semantic signals, while the structural component alone remains useful.

5.2 Threats to Validity

Construct Validity. A primary threat is whether our evaluation metrics adequately capture sampling quality. We mitigate this by combining six intrinsic metrics with downstream RCA accuracy. API Coverage and Path Coverage quantify coverage at the entry-point and execution-flow levels, Trace Pattern Coverage and Shannon Entropy quantify behavioral diversity, and Proportion Anomaly and Proportion Rare quantify whether diagnostically valuable traces are preserved.

Another concern is the choice of RCA tools. Several learning-based RCA tools performed poorly on our benchmark, likely due to domain shift from the simpler systems on which they were originally developed or evaluated. This is consistent with our previous findings on the generalization limits of learning-based RCA methods [7]. We therefore focus the RCA study on three tools that remain stable on our dataset, while retaining the intrinsic metrics as an independent view of sampling quality.

A further threat concerns the fidelity of Gleaner's semantic signals. EPS is intentionally lossy and does not preserve all timing or hierarchical information. However, EPS is used only as an

in-memory proxy for sampling decisions; once a trace is selected, the complete original trace, including timestamps, topology, and raw logs, is preserved for downstream analysis. Moreover, the results in §4.3 show that no evaluated system reaches 100% Trace Pattern Coverage even at a 10% sampling rate, indicating that the current EPS granularity is already sufficiently fine for the sampling task. Once the quota allocator has grouped traces by API endpoint, distinctions among independent events and causal relations become more important than further encoding timing or hierarchical details. In this sense, omitting those details is a deliberate trade-off for online efficiency rather than an accidental loss of information.

In addition, log parsing introduces uncertainty. We use Drain to remove dynamic fields and stabilize template IDs, since otherwise high-cardinality log contents would cause severe state-space explosion. Imperfect parsing may still over-merge or split templates. In our setting, Gleaner depends more on template stability than on perfect semantic parsing, because template IDs serve as lightweight event markers rather than full log interpretations.

Internal Validity. Implementation bias is a potential threat in comparative evaluation. We therefore use official baseline implementations when available and otherwise follow published specifications as closely as possible. For MicroRCA, Nezha, and ShapleyIQ, we retain the default algorithmic settings from their original releases; the only changes are engineering optimizations needed to execute the evaluation pipeline at scale, especially for Nezha, without altering the underlying ranking logic. To reduce variance from stochastic factors, all metrics are computed independently for each of the 161 fault-injection cases and then averaged. Our evaluation pipeline is open-sourced to support independent verification.

External Validity. Our study uses open-source benchmarks, which are necessarily simpler and cleaner than many production deployments. Although the cross-system evaluation demonstrates consistent performance across diverse architectures, the advantages of Gleaner are stronger in systems with multiple entry points, richer event spaces, and more complex trace patterns. Industrial systems may further exhibit noisier, delayed, or misconfigured alarms, incomplete logs, broken trace contexts, heterogeneous instrumentation quality, and more complex failure interactions. These factors may weaken the precision of alarm-driven allocation or reduce the benefit of semantic encoding. In such cases, Gleaner does not fail abruptly: missing logs and alarms reduce it toward a structure-driven diversity sampler, while noisy alarms are constrained by quota caps and the diversity-preserving selector so that no single alarmed group can dominate the budget and non-alarmed groups still retain baseline coverage. We partially address these concerns by evaluating the *w/o Logs w/o Alarms* variant on Dataset $\mathcal{B}$; the results in Figure 5 show that Gleaner’s structural encoding alone remains competitive with, and often superior to, strong trace-only baselines. Nevertheless, validation on industrial workloads remains necessary before making stronger claims about deployment-wide impact.

6 Related Work

Distributed Tracing Standards and Ecosystem. Standardization efforts have driven the evolution of distributed tracing toward semantic awareness. OpenTracing [30] and its popular implementation Jaeger [19] laid the groundwork by enabling log embedding within spans. The de facto standard, OpenTelemetry [27], advances this by not only unifying trace-log correlation [28] but also actively exploring how to incorporate intra-span events into sampling strategies [29]. Gleaner provides a concrete implementation for this vision. Its lightweight EPS representation encodes both trace structure and log semantics using simple hashable bigrams, offering a practical path for event-aware sampling without complex graph-based infrastructure.

Trace Sampling. Mainstream online sampling has evolved from simple probabilistic selection to sophisticated strategies. Early diversity-aware samplers like PEACH [21] use hierarchical clustering.

Anomaly-focused methods include **Sieve** [18], which uses RRCF to detect structural and temporal anomalies, and **Sifter** [22], which samples traces with high prediction error. **TraStrainer** [17] incorporates external metrics to prioritize traces correlated with system-level anomalies. The current state-of-the-art, **TracePicker** [39], formulates sampling as a two-phase optimization problem for coverage and latency preservation.

A distinct line of work employs GNNs to learn powerful trace representations, including STEAM [15], TraceCRL [46], and iTCRL [34]. Notably, iTCRL is also semantically aware by modeling log events as graph nodes. However, these approaches require offline training and incur high inference overhead, rendering them unsuitable for online sampling [9, 39]. Gleaner is designed specifically to overcome this trade-off, providing lightweight semantic awareness in a fully online model. An indirect comparison confirms Gleaner's superior balance of efficiency and effectiveness: TracePicker is shown to outperform STEAM [39], while our results in §4.3 show Gleaner surpasses TracePicker on the same public dataset.

Alternative Approaches. Beyond sampling, other paradigms address trace volume. **TraceZip** [6] uses lossless compression via structural deduplication, a technique orthogonal and potentially complementary to sampling strategies. Others propose architectural innovations: **Mint** [16] captures all traces by shifting from a keep-or-discard model to one based on commonality and variability, aggregating common templates while filtering parameters. **Hindsight** [47] implements retroactive sampling, analogous to a car's dash-cam, by logging all data locally and retrieving a full trace only after an anomaly is detected. These architectural concepts inspire future work in integrating Gleaner's principles into collection agents.

7 Conclusion

This paper introduces **Gleaner**, a semantically-aware trace sampler that resolves the tension between semantic richness and computational efficiency through lightweight EPS encoding, alarm-driven adaptive budget allocation, and DPP-based diversity optimization. Our evaluation demonstrates that Gleaner achieves superior coverage, diversity, and anomaly capture compared to state-of-the-art baselines, enabling RCA tools to achieve higher diagnostic accuracy—in some cases, even outperforming the complete unsampled dataset. This counter-intuitive result reveals that **strategic data curation can enhance signal quality beyond mere data quantity**. By bridging inter-span structure with intra-span semantics while maintaining computational tractability, Gleaner transforms trace sampling from passive volume reduction into active signal enhancement for downstream diagnostics. We provide an open-source benchmark dataset (161 fault cases, 1.4M+ traces, 125K events) to support future AIOps research, demonstrating that careful design enables sampling less data while providing better insights.

8 Data Availability

The Gleaner implementation is publicly available at [42], and the benchmark dataset introduced in this paper is publicly available at [41].

References

- [1] Chetan Bansal, Sundararajan Renganathan, Ashima Asudani, Olivier Midy, and Mathru Janakiraman. 2020. DeCaf: diagnosing and triaging performance issues in large-scale cloud services. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice* (Seoul, South Korea) (ICSE-SEIP '20). Association for Computing Machinery, New York, NY, USA, 201–210. doi:10.1145/3377813.3381353
- [2] Ivan Beschastnikh, Perry Liu, Albert Xing, Patty Wang, Yuriy Brun, and Michael D. Ernst. 2020. Visualizing Distributed System Executions. *ACM Trans. Softw. Eng. Methodol.* 29, 2, Article 9 (March 2020), 38 pages. doi:10.1145/3375633
- [3] Chaos Mesh Authors. 2025. Chaos Mesh: Chaos Engineering Platform for Kubernetes. <https://chaos-mesh.org/>.

- [4] Laming Chen, Guoxin Zhang, and Hanning Zhou. 2018. Fast greedy MAP inference for determinantal point process to improve recommendation diversity. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 5627–5638.
- [5] Yu Chen, Zhi-Ming Xiao, and Fei Teng. 2024. A Root Cause Localization Method Based on Event Call Chains for Microservices. In *2024 16th International Conference on Communication Software and Networks (ICCSN)*. 43–48. doi:10.1109/ICCSN63464.2024.10793332
- [6] Zhuangbin Chen, Junsong Pu, and Zibin Zheng. 2025. Tracezip: Efficient Distributed Tracing via Trace Compression. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA019 (June 2025), 23 pages. doi:10.1145/3728888
- [7] Aoyang Fang, Songhan Zhang, Yifan Yang, Haotong Wu, Junjielong Xu, Xuyang Wang, Rui Wang, Manyi Wang, Qisheng Lu, and Pinjia He. 2025. Rethinking the Evaluation of Microservice RCA with a Fault Propagation-Aware Benchmark. <https://arxiv.org/abs/2510.04711v2>
- [8] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: practical and scalable ML-driven performance debugging in microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 135–151. doi:10.1145/3445814.3446700
- [9] Fei Gao, Ruyue Xin, Xiaocui Li, and Yaqiang Zhang. 2025. Are GNNs Actually Effective for Multimodal Fault Diagnosis in Microservice Systems? . In *2025 IEEE International Conference on Web Services (ICWS)*. IEEE Computer Society, Los Alamitos, CA, USA, 127–129. doi:10.1109/ICWS67624.2025.00025
- [10] Shenghui Gu, Guoping Rong, Tian Ren, He Zhang, Haifeng Shen, Yongda Yu, Xian Li, Jian Ouyang, and Chunan Chen. 2023. TrinityRCL: Multi-Granular and Code-Level Root Cause Localization Using Multiple Types of Telemetry Data in Microservice Systems. *IEEE Transactions on Software Engineering* 49, 5 (2023), 3071–3088.
- [11] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1387–1397. doi:10.1145/3368089.3417066
- [12] Yongqi Han, Qingfeng Du, Ying Huang, Pengsheng Li, Xiaonan Shi, Jiaqi Wu, Pei Fang, Fulong Tian, and Cheng He. 2024. Holistic Root Cause Analysis for Failures in Cloud-Native Systems Through Observability Data. *IEEE Transactions on Services Computing* 17, 6 (2024), 3789–3802. doi:10.1109/TSC.2024.3478759
- [13] Yongqi Han, Qingfeng Du, Ying Huang, Jiaqi Wu, Fulong Tian, and Cheng He. 2024. The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 931–943. doi:10.1145/3691620.3695475
- [14] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. 2017. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In *2017 IEEE International Conference on Web Services (ICWS)*. 33–40. doi:10.1109/ICWS.2017.13
- [15] Shilin He, Botao Feng, Liqun Li, Xu Zhang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2023. STEAM: observability-preserving trace sampling. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1750–1761.
- [16] Haiyu Huang, Cheng Chen, Kunyi Chen, Pengfei Chen, Guangba Yu, Zilong He, Yilun Wang, Huxing Zhang, and Qi Zhou. 2025. Mint: Cost-Efficient Tracing with All Requests Collection via Commonality and Variability Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 683–697. doi:10.1145/3669940.3707287
- [17] Haiyu Huang, Xiaoyu Zhang, Pengfei Chen, Zilong He, Zhiming Chen, Guangba Yu, Hongyang Chen, and Chen Sun. 2024. TraStrainer: Adaptive Sampling for Distributed Traces with System Runtime State. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 473–493.
- [18] Zicheng Huang, Pengfei Chen, Guangba Yu, Hongyang Chen, and Zibin Zheng. 2021. Sieve: Attention-based Sampling of End-to-End Trace Data in Distributed Microservice Systems. In *2021 IEEE International Conference on Web Services (ICWS)*. 436–446. doi:10.1109/ICWS53863.2021.00063
- [19] Jaeger. 2025. *Jaeger: open source, distributed tracing platform*. Retrieved 2025-09-06 from <https://www.jaegertracing.io/>
- [20] Xinrui Jiang, Yicheng Pan, Meng Ma, and Ping Wang. 2023. Look Deep into the Microservice System Anomaly through Very Sparse Logs. In *Proceedings of the ACM Web Conference 2023*. 2970–2978.
- [21] Pedro Las-Casas, Jonathan Mace, Dorgival Guedes, and Rodrigo Fonseca. 2018. Weighted Sampling of Execution Traces: Capturing More Needles and Less Hay. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '18)*. Association for Computing Machinery, New York, NY, USA, 326–332. doi:10.1145/3267809.3267841
- [22] Pedro Las-Casas, View Profile, Giorgi Papakerashvili, View Profile, Vaastav Anand, View Profile, Jonathan Mace, and View Profile. 2019. Sifter. In *Proceedings of the ACM Symposium on Cloud Computing*. 312–324. doi:10.1145/3357223.

3362736

- [23] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.
- [24] Bowen Li, Xin Peng, Qilin Xiang, Hanzhang Wang, Tao Xie, Jun Sun, and Xuanzhe Liu. 2021. Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empir Software Eng* 27 (Nov. 2021), 25. doi:10.1007/s10664-021-10063-9
- [25] Ye Li, Jian Tan, Bin Wu, Xiao He, and Feifei Li. 2024. ShapleyIQ: Influence Quantification by Shapley Values for Performance Debugging of Microservices. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Vancouver, BC, Canada) (ASPLOS '23). Association for Computing Machinery, New York, NY, USA, 287–323. doi:10.1145/3623278.3624771
- [26] Hongyi Liu, Xiaosong Huang, Mengxi Jia, Tong Jia, Jing Han, Zhonghai Wu, and Ying Li. 2024. UAC-AD: Unsupervised Adversarial Contrastive Learning for Anomaly Detection on Multi-Modal Data in Microservice Systems. *IEEE Transactions on Services Computing* 17, 6 (2024), 3887–3900. doi:10.1109/TSC.2024.3411481
- [27] OpenTelemetry Community. 2025. *OpenTelemetry Concepts - Traces*. Retrieved 2025-09-08 from <https://opentelemetry.io/docs/concepts/signals/traces/>
- [28] OpenTelemetry Community. 2025. *OpenTelemetry Logging*. Retrieved 2025-09-06 from <https://opentelemetry.io/docs/specs/otel/logs/>
- [29] OpenTelemetry Community. 2025. *opentelemetry-specification/oteps/0265-event-vision.md at v1.48.0 · open-telemetry/opentelemetry-specification*. Retrieved 2025-09-06 from <https://github.com/open-telemetry/opentelemetry-specification/blob/v1.48.0/oteps/0265-event-vision.md>
- [30] OpenTracing Community. 2025. *OpenTracing Overview - Spans*. Retrieved 2025-09-06 from <https://opentracing.io/docs/overview/spans/>
- [31] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Technical Report. Google, Inc. <http://research.google.com/archive/papers/dapper-2010-1.pdf>
- [32] Chang-Ai Sun, Tao Zeng, Wanqing Zuo, and Huai Liu. 2023. A Trace-Log-Clusterings-Based Fault Localization Approach to Microservice Systems. In *2023 IEEE International Conference on Web Services (ICWS)*. 7–13. doi:10.1109/ICWS60048.2023.00013 ISSN: 2836-3868.
- [33] Yongqian Sun, Binpeng Shi, Mingyu Mao, Minghua Ma, Sibao Xia, Shenglin Zhang, and Dan Pei. 2024. ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1183–1194. doi:10.1145/3691620.3695495
- [34] Xiangbo Tian, Shi Ying, Tiangang Li, Mengting Yuan, Ruijin Wang, Yishi Zhao, and Jianga Shang. 2024. iTCRL: Causal-Intervention-Based Trace Contrastive Representation Learning for Microservice Systems. *IEEE Transactions on Software Engineering* 50, 10 (2024), 2583–2601. doi:10.1109/TSE.2024.3446532
- [35] Hanzhang Wang, Zhengkai Wu, Huai Jiang, Yichao Huang, Jiamu Wang, Selcuk Kopru, and Tao Xie. 2021. Groot: An event-graph-based approach for root cause analysis in industrial settings. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 419–429.
- [36] Yuqing Wang, Mika V. Mäntylä, Serge Demeyer, Mutlu Beyazit, Joanna Kisaakye, and Jesse Nyssölä. 2025. Cross-System Categorization of Abnormal Traces in Microservice-Based Systems via Meta-Learning. *Proc. ACM Softw. Eng.* 2, FSE (2025), FSE027:576–FSE027:598. doi:10.1145/3715742
- [37] Yidan Wang, Zhouruixing Zhu, Qiui Fu, Yuchi Ma, and Pinjia He. 2024. MRCA: Metric-level Root Cause Analysis for Microservices via Multi-Modal Data. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1057–1068. doi:10.1145/3691620.3695485
- [38] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*. 1–9. doi:10.1109/NOMS47738.2020.9110353
- [39] Shuaiyu Xie, Jian Wang, Maodong Li, Peiran Chen, Jifeng Xuan, and Bing Li. 2025. TracePicker: Optimization-Based Trace Sampling for Microservice-Based Systems. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE081 (June 2025), 22 pages. doi:10.1145/3729351
- [40] Junjieloung Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures? [https://opve15\(vi](https://opve15(vi)

- [42] Yifan Yang. 2026. Gleaner: Implementation and Artifacts. <https://github.com/OperationsPAI/Gleaner>. Accessed: 2026-04-18.
- [43] Zhenhe Yao, Changhua Pei, Wenxiao Chen, Hanzhang Wang, Liangfei Su, Huai Jiang, Zhe Xie, Xiaohui Nie, and Dan Pei. 2024. Chain-of-Event: Interpretable Root Cause Analysis for Microservices through Automatically Learning Weighted Event Causal Graph. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (Porto de Galinhas, Brazil) (FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 50–61. doi:10.1145/3663529.3663827
- [44] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 553–565. doi:10.1145/3611643.3616249
- [45] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-based Deep Learning. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 623–634. doi:10.1145/3510003.3510180
- [46] Chenxi Zhang, Xin Peng, Tong Zhou, Chaofeng Sha, Zhenghui Yan, Yiru Chen, and Hong Yang. 2022. TraceCRL: contrastive representation learning for microservice trace analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1221–1232. doi:10.1145/3540250.3549146
- [47] Lei Zhang, Zhiqiang Xie, Vaastav Anand, Ymir Vigfusson, and Jonathan Mace. 2023. The Benefit of Hindsight: Tracing Edge-Cases in Distributed Systems. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 321–339. <https://www.usenix.org/conference/nsdi23/presentation/zhang-lei>
- [48] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibao Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, Dai Zhang, Zhenyu Zhu, and Dan Pei. 2023. Robust Failure Diagnosis of Microservice System Through Multimodal Data. *IEEE Transactions on Services Computing* 16, 6 (2023), 3851–3864. doi:10.1109/TSC.2023.3290018
- [49] Shenglin Zhang, Sibao Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2025. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). doi:10.1145/3715005 Just Accepted.
- [50] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Yan Chaoran, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, and Bo Zhang. 2024. mABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 4017–4033. doi:10.18653/v1/2024.findings-emnlp.232
- [51] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2018. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering* 47, 2 (2018), 243–260.
- [52] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* 47 (Feb. 2021), 243–260. doi:10.1109/TSE.2018.2887384 Conference Name: IEEE Transactions on Software Engineering.
- [53] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 323–324. doi:10.1145/3183440.3194991